

pttai: Build-Time Dataflow Validation for LLM Pipeline Graphs

Anonymous

Abstract

LLM agent pipelines are graphs of model calls, tools, and branches over a shared state. A mis-wired one—a node that reads a state key nothing produced, an unwired branch, a concurrent write to a key with no reducer—does not fail when you build it. It fails *after* the model has already run, once execution reaches the broken node, having burned tokens, latency, and (with tools) real side effects on a run that was doomed at construction. We present pttai, a thin declarative layer that compiles a >-wired pipeline to a native LangGraph StateGraph and, before compiling, runs a forward *dataflow* analysis over the graph. To our knowledge this is the first compile-time dataflow analysis for LLM agent graphs. It is a MAY/MUST fixpoint in the tradition of definite-assignment analysis: hard errors are raised only from the MAY (some-path) set, so the analysis is false-positive-free by construction—it never rejects a graph that would have run. On a 15-item bug benchmark pttai fails the build on all 15; 13 of these raw LangGraph surfaces only at run-time or never, costing 15 wasted model calls against pttai’s zero, with no false positives on 19 valid pipelines. The demonstration is an interactive playground that paints the offending node red at build time, before any model call.

1 Introduction

Language-model pipelines have become graphs. A retrieval-augmented QA system retrieves passages, then answers grounded in them; an extract-then-summarize pipeline pulls typed fields from a ticket, then writes a one-line triage summary from those fields; a document-triage pipeline classifies an incoming message and routes it to a specialized handler. Frameworks such as LangGraph (LangChain, 2024) express these as explicit state-machine graphs: nodes read and write a shared state object, edges (some conditional) move control between them.

These graphs fail differently from ordinary programs. In a shared-state graph, a node that reads state["summary"] before any node has written summary does not raise when the graph is built—it raises a `KeyError` at run time, once control reaches that node. By then the pipeline has already made model calls: tokens spent, latency incurred, and, if earlier nodes called tools, side effects committed, all on a run that was structurally broken before it started. The same is true of an unwired conditional branch (a routing key that resolves to nowhere) and of two parallel branches writing the same non-reduced state key (a runtime `InvalidUpdateError`). We verified that all three are recurring, publicly reported failures against raw LangGraph (§4).

The root cause is that the graph builder checks *structure* but not *dataflow*: LangGraph’s `compile()` does not verify that every state key a node reads is produced upstream. Ordinary compilers have solved the analogous problem for decades—definite-assignment analysis rejects a Java program that reads a local before it is assigned on some path (Gosling et al., 2014; Kildall, 1973; Aho et al., 2006)—but no LLM-graph framework applies it to the graph’s shared state.

We present pttai, a declarative layer over LangGraph whose contribution is exactly this missing check.¹ Our contributions are:

1. **A compile-time dataflow analysis for LLM agent graphs** (§2.2): a forward MAY/MUST fixpoint over the wired graph that fails the build—with zero model calls—when a node reads a key no upstream node produces, when a branch is unwired, or when parallel branches write an unreduced key. Because hard errors come only from the MAY (some-path) set, the analysis is *false-positive-free* by

¹Source, examples, benchmark, and demo: <https://github.com/TeeratP/agent-ic-framework>.



Figure 1: The `>` DSL builds a linked node structure in memory; `AgenticGraph` walks it into `add_node/add_edge/Send` calls; the build-time dataflow validator gates the build (`GraphValidationError` on failure, *before any model call*); on success it compiles to a native `LangGraph StateGraph`.

construction: a graph that would actually run is never rejected.

2. **An interactive demonstration (§3)**: a playground in which you edit a pipeline and see the compiled `LangGraph`, or, when the build fails, the pre-compile graph with the offending node(s) painted red—the picture of the bug, before any model call.
3. **An evaluation (§4)** on a bug benchmark (13 differentiating bug classes caught at build with 0 false positives on 19 valid pipelines), a wild-bug survey grounding the taxonomy, and negligible build overhead with identical run-time behavior.

pttai borrows the “ergonomic layer over a powerful runtime” idea from Keras (Chollet et al., 2015), but the ergonomics are not the contribution; the validator is. Everything pttai builds is a plain `LangGraph` graph, so streaming, async, checkpointers, and observability are inherited, not reimplemented.

2 System Design

2.1 The `>` DSL and compilation

A pttai pipeline is built by instantiating `Node` objects and wiring them with `>` (Figure 1). Wiring is deferred: `a > b` sets `a.child = b` and returns `b`, so `a > b > c` builds a linked structure in memory—no graph edges yet. `AgenticGraph(start, end_nodes)` then walks that structure and emits the actual `LangGraph` `add_node/add_edge/add_conditional_edges` calls before compiling. There are three node types: `AgentNode` (a system-prompted model call with an internal tool-call loop), `DecisionNode` (constrained structured-output routing: the model must return one of the declared choices, wired `decision["c"] > handler`), and `InputNode` (human-in-the-loop via `LangGraph`’s `interrupt`). Fan-out is `fanout(a, b)` and `map-reduce` is

`.map(field)`.

Nodes read and write a shared, reducer-based state. Each node declares its reads and writes; a node returns only the keys it updates and the channel reducers merge them, which is what makes parallel branches and checkpointing correct. Crucially, an `AgenticGraph` *is* a `LangGraph StateGraph` and its compiled artifact is `LangGraph`’s `CompiledStateGraph`: pttai adds a build pass on top of `LangGraph` and then hands the compiled graph to it. Nothing in the runtime is reimplemented.

2.2 The build-time dataflow validator

When `AgenticGraph(...)` is constructed, it runs a static analysis over the wired nodes and **fails the build** on a hard error, raising `GraphValidationError` before `compile()`. This is the contribution.

The analysis. During the build we record a tagged edge list: each edge is (`src`, `dst`, `kind`) where `kind` is `seq` (an unconditional, AND edge) or `cond` (an exclusive, OR edge: a `DecisionNode` choice or a `Send` fan-out target). Over this graph we compute, per node, two sets of available state keys by forward fixpoint:

- **MAY**—keys available on *some* path into the node. A monotone-*increasing* union fixpoint.
- **MUST**—keys guaranteed on *every* path into the node. A monotone-*decreasing* fixpoint.

The transfer function is the standard forward one—a node’s out-set is its in-set plus the keys it writes—with two combining rules at joins that distinguish this from a scalar CFG analysis: *union across an AND-parallel join* (a node after `fanout(a,b)` sees both branches’ writes), and *intersect across an exclusive OR-group* (a node after a `DecisionNode` is guaranteed only the keys every choice branch produces). The seed is the *input* keys—reduced channels, schema keys no node writes, and user-declared inputs—not all declared keys; seeding with inputs rather than the whole schema is what lets the analysis catch a read of a *computed* key before its producer has run. The fixpoint iterates until a full sweep changes nothing, bounded by the node count.

Soundness / no false positives. Hard *errors* are raised only from the **MAY** set: a read is an error only when the key is available on *no* path into the node. If a key is available on some but not all paths

Error (fails build)	Condition
read-before-write	reads a computed key produced only downstream / on a sibling branch (MAY)
undeclared key	reads/writes a key absent from the state schema
concurrent write, no reducer	two parallel branches write the same non-reduced key
dangling choice	a DecisionNode choice with no wired handler
dead-end node	a non-terminal node with no child
duplicate node name	two distinct nodes share a name [†]
prompt/read mismatch	a node_prompt {placeholder} that is not a declared scalar read
Warning (build succeeds)	Condition
some-paths read optional read unmet	key in MAY but not MUST reads=["x?"] never produced
unreachable node	not reachable from the start node
end node w/ children	a declared end node still has a child

Table 1: The validator’s error and warning classes. [†]Duplicate names and dead ends are raised structurally during the build walk (as ValueError); the rest come from the dataflow pass. A dead read (a scalar read never interpolated) is an additional warning.

(MAY but not MUST), that is a *warning*, never a build failure. So the validator never rejects a graph that would actually run—false-positive-freedom is a property of raising errors from the some-path set, exactly as definite-assignment analysis reports “used before assigned” only when *no* path assigns the variable (Gosling et al., 2014; Nielson et al., 1999). The MUST set only drives warnings.

What it checks. On top of the dataflow sets, pttai raises seven error classes and four warning classes (Table 1). Errors fail the build; warnings surface in a model.summary()-style table and can be promoted to errors with strict=True.

3 Demonstration

The demonstration is a browser playground (Figure 2). A reviewer edits a small pttai snippet—one of the shipped NLP pipelines (RAG QA, extract→summarize, document triage) or their own—and submits it. On a clean build the playground renders the *compiled LangGraph* as a node/edge diagram plus the validator’s summary() table (every node’s reads, writes, and available

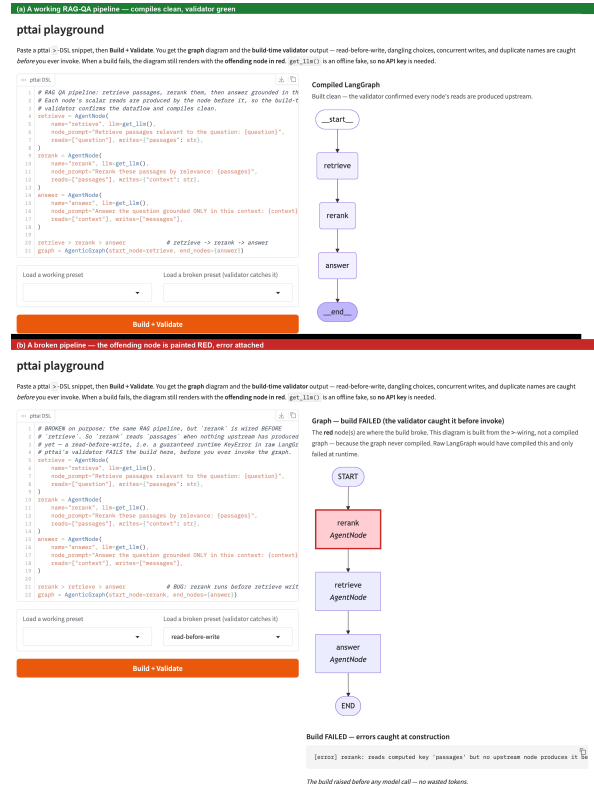


Figure 2: The playground. A broken pipeline is rejected at build time and drawn with the offending node in red and the validator error attached—no model call is made.

keys). The intended live moment: take a working RAG-QA pipeline and reorder two nodes so the answer step reads passages before the retriever runs. Raw LangGraph would compile this and only fail after the retrieval model call. pttai instead rejects it at build: the diagram is redrawn from the pre-compile wiring with the **offending node painted red** and the exact error attached (“reads computed key ‘passages’ but no upstream node produces it”), with *zero model calls made*. The reviewer sees the picture of the bug before running anything.

Because the playground execs user-supplied code, it is sandboxed for public hosting: the snippet is size-capped, AST-checked (imports allow-listed to pttai/typing/safe stdlib; dunder access and dangerous builtins such as eval/open/__import__ rejected), run with a restricted __builtins__, and executed under a per-request timeout. The model backend is an offline fake, so the demo needs no API key and makes no network call. A <2.5-minute screencast accompanies the submission.

4 Evaluation

All evaluations run offline (no API key); the harness is in the repository under `eval/`.

Bug benchmark. We built bugbench, 15 buggy pipelines (three each of read-before-write, dangling choice, dead-end; two each of concurrent unreduced write, prompt/read mismatch; two duplicate-name) and 19 valid pipelines (the shipped example suite). For each we record whether pttai fails the build and how many model calls each framework wastes before the bug surfaces. pttai fails the build on **all 15** buggy pipelines with the correct diagnostic, at a median of under 1 ms and **0** wasted model calls. Raw LangGraph catches only the 2 duplicate-name cases at build; the other **13** it surfaces at run time (10 as `KeyError` / `InvalidUpdateError` after the model has run) or never (3 dead-end cases loop silently). Across the buggy set, LangGraph wastes **15** model calls to pttai’s **0**. On the 19 valid pipelines pttai raises **0 false positives** (0 spurious build failures)—consistent with the some-path error rule. So on the differentiating classes the catch rate is 13/13 with 0 false positives; duplicate names are caught by both frameworks and are not a differentiator.

Externally-grounded study (in progress). A benchmark authored by us invites the objection that the validator catches only the bugs its author planted. We therefore designed a complementary study (`eval/llmgen/`) in which a frontier model generates pipelines from short NLP task specs—given the docs and *never told to write bugs*—and every generated pipeline is built and adjudicated, so the bug distribution comes from real AI-written agent code rather than a hand-picked corpus. The scoring path is proven offline on a committed sample set of 9 pipelines (4 clean, 5 buggy): 5 flagged, **0 false positives**, of which raw LangGraph would surface 4 only at run time and 1 silently. **The headline numbers require an API key and a generation run that the build environment cannot perform; they are reported as a placeholder and must be filled from a keyed run before submission.**

Wild-bug survey. To show the taxonomy is not author-imagined, we mined public issue trackers for each targeted class (Table 2): nine independent, verified reports against raw LangGraph, at

Bug class (runtime symptom)	Reports
read-before-write (<code>KeyError</code> on a state key)	2
concurrent write, no reducer (<code>InvalidUpdateError</code>)	3
dangling conditional edge	2
dead-end / non-terminating graph	2
Total	9

Table 2: Independent, publicly reported wild bugs against raw LangGraph, by targeted class (details and issue links in the repository’s docs/`EVIDENCE.md`).

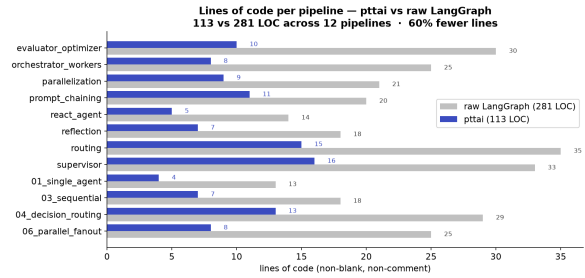


Figure 3: Lines of code, pttai vs. hand-written LangGraph, across 12 canonical pipelines (113 vs. 281 lines total).

least two per class. Every one is a runtime failure or a non-terminating graph that pttai’s build-time check would have caught first. (Duplicate names and the prompt/read check are excluded here: the former both frameworks already catch at build, and the latter is a pttai-internal reformulation of a generic `str.format` error with no LangGraph-specific report.)

Conciseness (a side effect). Across 12 canonical pipelines (the agentic-pattern and basics suites) ported both ways, the pttai versions total 113 lines against 281 for hand-written LangGraph—about 60% fewer (Figure 3). This is a consequence of folding `add_node` / `add_edge` / structured-output routing / the tool-call loop into the node abstractions, not the point of the system; the validator is.

Overhead. The added build pass (wiring + full dataflow validation + compile) has a sub-millisecond median and is within measurement noise of a bare LangGraph `compile()`; because pttai compiles to a native `CompiledStateGraph`, run time is LangGraph—identical model-call count and identical output.

5 Related Work

pttai is complementary to, not a replacement for, existing frameworks; it adds a static dataflow check none of them performs and compiles down to LangGraph. **LangGraph** (LangChain, 2024) is the runtime pttai targets; its `compile()` runs basic structural checks (it raises on a node with no path to the end) but does not analyze which state keys are produced before they are read—the gap this work fills. **DSPy** (Khattab et al., 2024) optimizes prompts and demonstrations against a metric; its control flow is arbitrary Python, so there is no declared graph to check statically. **LMQL** (Beurer-Kellner et al., 2023) and **Guidance** (Guidance AI, 2023) constrain *within* a single generation (decoding, output structure) rather than across a multi-node graph. **CrewAI** (CrewAI Inc., 2024) orchestrates via roles and a sequential/hierarchical process, with no declared node/edge graph or dataflow validation. **AutoGen**’s GraphFlow (Wu et al., 2023) and **LlamaIndex** Workflows (Liu and LlamaIndex, 2024) do validate graph *structure* (entry points, reachability, event-type wiring) before running, and **Haystack** (deepset, 2024) type-checks explicit component connections—but none performs a shared-state *dataflow* analysis (read-before-write / concurrent-unreduced-write), and thus none has a false-positive guarantee to offer. To our knowledge pttai is the only one of these that runs a build-time state-key dataflow analysis, and the only one that lowers to an established graph runtime rather than shipping its own.

Limitations

The dataflow analysis reasons about state-key *availability*, not *types or values*: it verifies that a key a node reads is produced upstream, not that its value has the expected type or is non-empty. A key produced with the wrong type still passes. The **MUST** (all-paths) set over-approximates exclusivity in one case—a decision handler that merges back via an unconditional edge unions into the successor’s **MUST** set as if unconditional—which can only *under-warn* (miss a some-paths warning); it never produces a wrong hard error, since errors come from **MAY** only. The analysis is intentionally sound-for-errors rather than complete: it will not flag a semantic bug that is nonetheless well-formed dataflow (e.g., a prompt that asks for the wrong thing). The wild-bug survey is a targeted

sample of public reports, not an exhaustive census, and establishes that the classes occur, not their frequency. Finally, the externally-grounded LLM-generation study is designed and its scoring path verified, but its headline numbers await a keyed generation run (§4); we do not report them here and do not claim them.

Ethics and Broader Impact

pttai’s goal is to catch broken agent pipelines *before* they run, which directly reduces wasted model calls—and thus the tokens, cost, latency, and energy spent on runs that were doomed at build time—and avoids committing tool side effects on such runs. The public playground execs user-submitted code; we sandbox it (allow-listed imports, restricted builtins, AST rejection of dangerous calls, size and time caps, offline fake model) so that hosting it does not expose the host, but we note that exec-based sandboxes are best-effort and a hardened deployment should add OS-level isolation. The validator makes no claim about the *safety, factuality, or bias of model outputs*; it checks pipeline dataflow, not what the models say. pttai is released under the MIT license.

References

- Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools*, 2nd edition. Addison-Wesley.
- Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting is programming: A query language for large language models. In *Proceedings of the ACM on Programming Languages (PLDI)*. ArXiv:2212.06094.
- François Chollet et al. 2015. Keras. <https://keras.io>.
- CrewAI Inc. 2024. CrewAI: Framework for orchestrating role-playing, autonomous ai agents. <https://github.com/crewAIInc/crewAI>. Docs: <https://docs.crewai.com/>. Accessed 2026.
- deepset. 2024. Haystack: An open-source framework for building production-ready llm applications. <https://github.com/deepset-ai/haystack>. Docs: <https://docs.haystack.deepset.ai/>. Accessed 2026.
- James Gosling, Bill Joy, Guy L. Steele, Gilad Bracha, and Alex Buckley. 2014. *The Java Language Specification, Java SE 8 Edition*. Addison-Wesley. Chapter 16, Definite Assignment.

Guidance AI. 2023. Guidance: A guidance language for controlling large language models. <https://github.com/guidance-ai/guidance>. Accessed 2026.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*. ArXiv:2310.03714.

Gary A. Kildall. 1973. A unified approach to global program optimization. In *Proceedings of the 1st ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 194–206.

LangChain. 2024. LangGraph: Low-level orchestration framework for stateful, multi-actor agent applications. <https://github.com/langchain-ai/langgraph>. Documentation: <https://langchain-ai.github.io/langgraph/>. Accessed 2026.

Jerry Liu and LlamaIndex. 2024. LlamaIndex workflows. <https://developers.llamaindex.ai/python/workflows/>. Accessed 2026.

Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. 1999. *Principles of Program Analysis*. Springer.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*. Preprint, arXiv:2308.08155. GraphFlow: <https://microsoft.github.io/autogen/>.

A Reproducing the evaluation

All numbers are produced offline (no API key) from the repository: `eval/bugbench/run.py` regenerates `eval/bugbench/results.csv` (the 15 buggy / 19 valid outcomes and per-pipeline wasted-call counts); `eval/loc_compare.py` regenerates `eval/loc_results.csv` (the 113 vs. 281 line counts over 12 pipelines); `eval/overhead.py` prints the build-time and runtime-parity measurements; and `eval/llmgen/score.py -gen-dir` samples runs the sample-set self-test of the externally-grounded study. The wild-bug reports with issue links are in `docs/EVIDENCE.md`.

B Example: a validated NLP pipeline

The extract-then-summarize pipeline. The extract node returns typed structured output; the summarize node reads those scalar fields and the validator confirms, at build time, that each is produced upstream before it is read.

```
extract = AgentNode(llm=llm,
node_prompt="Extract fields: {ticket}",
reads=["ticket"],
writes={"product": str, "severity":
int})
summarize = AgentNode(llm=llm,
node_prompt="severity-{severity} on
{product}",
reads=["product", "severity"],
writes=["summary"],
output_field="summary")
extract > summarize
AgenticGraph(start_node=extract,
end_nodes={summarize})
```

Swapping the two nodes (`summarize > extract`) makes summarize read product and severity before extract produces them; the build fails with a read-before-write error and no model call is made.